

ISYE 6740 Summer 2026

Homework 5

(100 points)

Student Name Here

Provided Data and Submission Requirements:

- All results, explanations, and math must be included in your PDF submission. Handwritten work is not accepted per the syllabus. Screenshots of work is also not accepted.
- If applicable, questions marked with **GS** must be submitted to Gradescope. Failure to pass gradescope tests will result in penalties to the points for that question.

This assignment does not have any Gradescope requirements

- Q3: Medical imaging reconstruction [cs.mat]
- Q4: AdaBoost figure [hw.png]
- Q5: Random forest and one-class SVM for email spam classifier [spambase.data]
- Q6: Locally weighted linear regression [data.mat]

As usual, please submit a report with sufficient explanation of your answers to each the questions, together with your code, in a zip folder.

1. Concept Questions (5 points)

An average of B i.i.d. random variables, each with variance σ^2 , has variance $\frac{1}{B}\sigma^2$. If the variables X_i are simply identically distributed (i.d.) but not necessarily independent, with positive pairwise correlation $\rho := \text{Cov}(X_i, X_j)/\sigma^2$, where $\sigma^2 = \text{Var}(X_i)$, show that the variance of the average $\bar{X} := \frac{1}{B} \sum_{i=1}^B X_i$, is given by

$$\text{VAR}(\bar{X}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

To help understand why this is an important observation for Random Forest, explain why you would want to de-correlate the trees.

2. Bayes classifier optimality (10 points)

Prove that the Bayes classifier is optimal in that it minimizes the Bayes risk. Let $X \in \mathbb{R}^d$ be a feature vector, and $Y \in \{1, 2, \dots, K\}$ be the class label. Let $P(Y = k|X)$ be the posterior probability of class k given the feature X . Let $h(X)$ be any classifier mapping X to one of the class labels. Define the risk of the classifier as (the probability of mis-classification):

$$R(h) = P(h(X) \neq Y).$$

Then show that the Bayesian classifier h^* (defined by picking the maximum posterior) achieves the minimum risk among all classifiers

$$h^* = \arg \min_h R(h)$$

Hint: Note that

$$R(h) = P(h(X) \neq Y) = 1 - P(h(X) = Y)$$

Then note that

$$P(h(X) = Y) = \mathbb{E}_X[\mathbb{E}_{Y|X}[\mathbb{I}\{h(X) = Y|X\}]],$$

where the expectation is taken with respect to X , and the conditional expectation, respectively; and $\mathbb{I}$ denotes the indicator function.

3. Medical imaging reconstruction (20 points)

In this problem, you will consider an example that resembles medical imaging reconstruction in MRI. We begin with a true image of dimension 50×50 (i.e., there are 2500 pixels in total). The data file is `cs.mat`; you can plot it first to see what it looks like. This image is truly sparse, in the sense that 2084 of its pixels have a value of 0, while 416 pixels have a value of 1. You can think of this image as a toy version of an MRI image that we are interested in collecting.

Because of the nature of the machine that collects the MRI image, it takes a long time to measure each pixel value individually, but it is faster to measure a linear combination of pixel values. We measure $n = 1300$ linear combinations, with the weights in the linear combination being random and independently distributed as $\mathcal{N}(0, 1)$. Because the machine is not perfect, we do not get to observe this directly, but we observe a noisy version. These measurements are given by the entries of the vector

$$y = Ax + \epsilon,$$

where $y \in \mathbb{R}^{1300}$, $A \in \mathbb{R}^{1300 \times 2500}$, and $\epsilon \sim \mathcal{N}(0, 25 \times I_{1300})$ where I_n denotes the identity matrix of size $n \times n$. In this homework, you can generate the data y using this model.

Now the question is: can we model y as a linear combination of the columns of A to recover some coefficient vector that is close to the image? Roughly speaking, the answer is yes.

Key points here: although the number of measurements $n = 1300$ is smaller than the dimension $p = 2500$, the true image is sparse. Thus, we can recover the sparse image using few measurements exploiting its structure. This is the idea behind the field of *compressed sensing*.

The image recovery can be done using lasso

$$\min_x \|y - Ax\|_2^2 + \lambda \|x\|_1.$$

1. (10 points) Use lasso to recover the image and select λ using 10-fold cross-validation. Plot the cross-validation error curves and show the recovered image using your selected lambda values.
2. (10 points) To compare, also use ridge regression to recover the image:

$$\min_x \|y - Ax\|_2^2 + \lambda \|x\|_2^2.$$

Select λ using 10-fold cross-validation. Plot the cross-validation error curves and show the recovered image using your selected lambda values. Which approach gives a better recovered image?

4. AdaBoost (25 points)

Consider the following dataset, plotted in the following figure. The first two coordinates represent the value of two features, and the last coordinate is the binary label of the data.

$$X_1 = (-1, 0, +1), X_2 = (-0.5, 0.5, +1), X_3 = (0, 1, -1), X_4 = (0.5, 1, -1),$$

$$X_5 = (1, 0, +1), X_6 = (1, -1, +1), X_7 = (0, -1, -1), X_8 = (0, 0, -1).$$

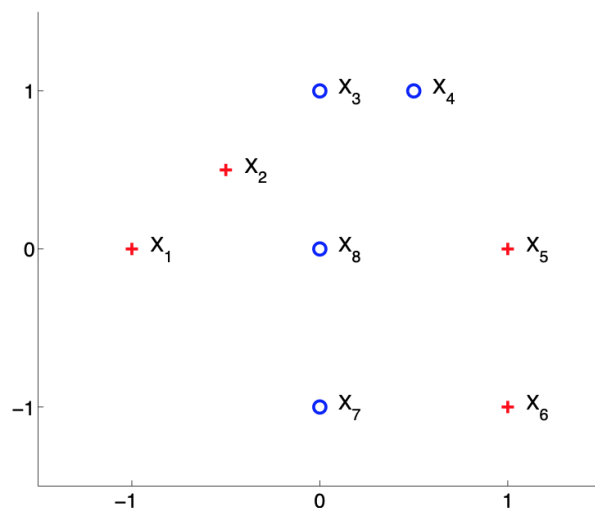


Figure 1: A small dataset for binary classification with AdaBoost.

In this problem, you will run through $T = 3$ iterations of AdaBoost with decision stumps (as explained in the lecture) as weak learners.

- (15 points) For each iteration $t = 1, 2, 3$, compute ϵ_t , α_t , Z_t , D_t mathematically by hand, showing all of your calculation steps. Draw the decision stumps on the figure. You can draw this by hand, plot using code, or a tool like Desmos. Fill in the following results table with the values of your AdaBoost parameters.

Table 1: Values of AdaBoost parameters at each timestep.

t	ϵ_t	α_t	Z_t	$D_t(1)$	$D_t(2)$	$D_t(3)$	$D_t(4)$	$D_t(5)$	$D_t(6)$	$D_t(7)$	$D_t(8)$
1											
2											
3											

- (10 points) What is the training error of this final AdaBoost model? Give a short explanation for why AdaBoost outperforms a single decision stump.

5. Random forest and one-class SVM for email spam classifier (20 points)

Your task for this question is to build a spam classifier using the UCI email spam dataset (source: <https://archive.ics.uci.edu/ml/datasets/Spambase>). This dataset came from the postmaster and individuals who had filed spam. **Please use the data file `spambase.data` provided in the assignment.** Headers are not necessary for this problem, but can be obtained by the `spambase.names` file from the source url. The collection of non-spam emails came from filed work and personal emails, and hence the word 'george' and the area code '650' (Palo Alto, CA) are indicators of non-spam. These are useful when constructing a personalized spam filter. You are free to choose any package for this homework. Note: there may be some missing values. You can just fill in zero. You should use the same train-test split for every part.

1. (5 points) Build a CART model and visualize the fitted classification tree. Please adjust the plot size/font as appropriate to ensure it is legible. Pruning this tree is not required, and if done reasoning should be stated as to why.
2. (5 points) Now, also build a random forest model. Randomly shuffle the data and partition to use 75% for training and the remaining 25% for testing. Compare and report the test error for your classification tree and random forest models on testing data. Plot the curve of test error (total misclassification error rate) versus the number of trees for the random forest, and plot the test error for the CART model (which should be a constant with respect to the number of trees).
3. (5 points) Fit a series of random-forest classifiers to the data to explore the sensitivity to the parameter ν (the number of variables selected at random to split). Plot both the OOB error as well as the test error against a suitably chosen range of values for ν .
4. (5 points) Now, we will use a one-class SVM approach for spam filtering. Randomly shuffle the data and partition to use 75% for training and the remaining 25% for testing. Extract all *non-spam* emails from the training block (75% of data you have selected) to build the one-class kernel SVM using RBF kernel. Then apply it to the 25% of data reserved for testing (thus, this is a novelty detection situation), and report the total misclassification error rate on these testing data. Tune your models appropriately to achieve good performance, i.e. by tuning the kernel bandwidth or other parameters. Give a short explanation on how you reached your final error rate and whether you feel this is a good model.

6. Locally weighted linear regression and bias-variance tradeoff (20 points)

The idea of locally weighted linear regression is that we will give more weight to data points in the training data that are close to the point at which we want to make a prediction. This can improve the bias but will also face a bias-variance tradeoff. This homework question is designed to look into this problem. For the coding portion, Any packages can be used.

Denote data point as (x_i, y_i) , $x_i \in \mathbb{R}^p$, $i = 1, \dots, n$. Given a Gaussian kernel function

$$K_h(z) = \frac{1}{(\sqrt{2\pi}h)^p} e^{-\frac{\|z\|^2}{2h^2}}, \quad z \in \mathbb{R}^p.$$

Local linear regression solves $\beta_0 \in \mathbb{R}$, $\beta_1 \in \mathbb{R}^p$, for a given predictor $x \in \mathbb{R}^p$:

$$\hat{\beta} := (\hat{\beta}_0, \hat{\beta}_1) = \arg \min \sum_{i=1}^n (y_i - \beta_0 - (x - x^i)^T \beta_1)^2 K_h(x - x_i)$$

1. (10 points) Show that the solution is given in the form

$$\hat{\beta} = (X^T W X)^{-1} X^T W Y$$

for properly defined X , W , and Y . Your final derivation must fully and clearly define your X , W , and Y vectors/matrices.

2. (5 points) Use all data in the `data.mat` file to perform local linear weighted linear regression. Do not split your data into train/test. Using 5-fold cross validation to tune the bandwidth parameter h , report a **plot** showing your cross validation curve and provide your optimal bandwidth, h .
3. (5 points) Use the tuned hyper-parameter h to make a prediction for $x = 1.4$. Provide the **predicted y value**. Show a **plot** showing your training data, prediction curve, and a marker indicating your prediction.

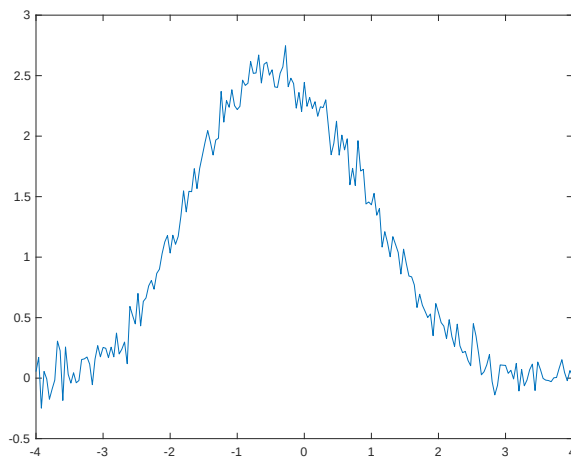


Figure 2: Illustration of noisy curve.